

Signals Lab 1 - Introduction to Python, Signals, and Audio

Colden Briggs - Eben Radocchia

Instructor: Dr. Ossareh

University of Vermont

September 21, 2026

Contents

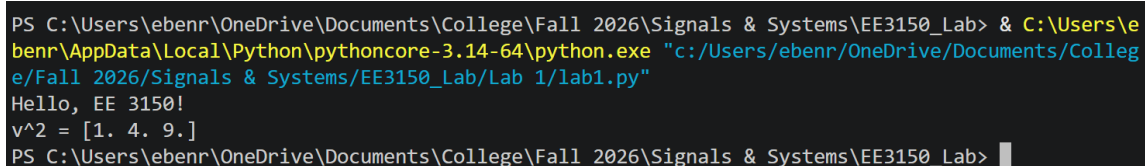
Introduction	2
Task 0.1	2
Task 1.1	3
Effect of a	4
Envelope vs oscillation rate	5
Task 2.1	6
Hand-drawn solutions	6
Stem plots	7
Comparison	7
Compression	7
y4 transform	7
Task 3.1	9
Aliasing	9
Beats	11
Plucked string	14
Conclusion	15

Introduction

This lab was centered around becoming familiar using and plotting signals in python. It consists of three main sections, each targeted at something different. The first section involved plotting different forms of functions, such as real and complex exponentials, square waves, and triangular waves, and seeing how changing function parameters affected the output. The second section was focused around signal transformations, and compared hand-drawn plots to computer-generated plots. The third and final section was focused around how sampling rate affects the output signal, how beats are introduced to signals, and imitating signal decay, which made use of some concepts introduced in earlier sections, such as signal transformations and real exponential envelopes.

Task 0.1

Figure 1 shows the output of the console for task 0.1.

A screenshot of a Windows command prompt window. The title bar is not visible. The prompt shows the current directory as 'PS C:\Users\ebenr\OneDrive\Documents\College\Fall 2026\Signals & Systems\EE3150_Lab'. The user has entered a command to run a Python script: '& C:\Users\ebenr\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/ebenr/OneDrive/Documents/College/Fall 2026/Signals & Systems/EE3150_Lab/Lab 1/lab1.py"'. The output of the script is displayed: 'Hello, EE 3150!' followed by 'v^2 = [1. 4. 9.]'. The prompt returns to 'PS C:\Users\ebenr\OneDrive\Documents\College\Fall 2026\Signals & Systems\EE3150_Lab>'.

```
PS C:\Users\ebenr\OneDrive\Documents\College\Fall 2026\Signals & Systems\EE3150_Lab> & C:\Users\ebenr\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/ebenr/OneDrive/Documents/College/Fall 2026/Signals & Systems/EE3150_Lab/Lab 1/lab1.py"
Hello, EE 3150!
v^2 = [1. 4. 9.]
PS C:\Users\ebenr\OneDrive\Documents\College\Fall 2026\Signals & Systems\EE3150_Lab>
```

Figure 1: Python Output

Task 1.1

Figure 2 shows the 2×2 figure including the triangular wave x_4 .

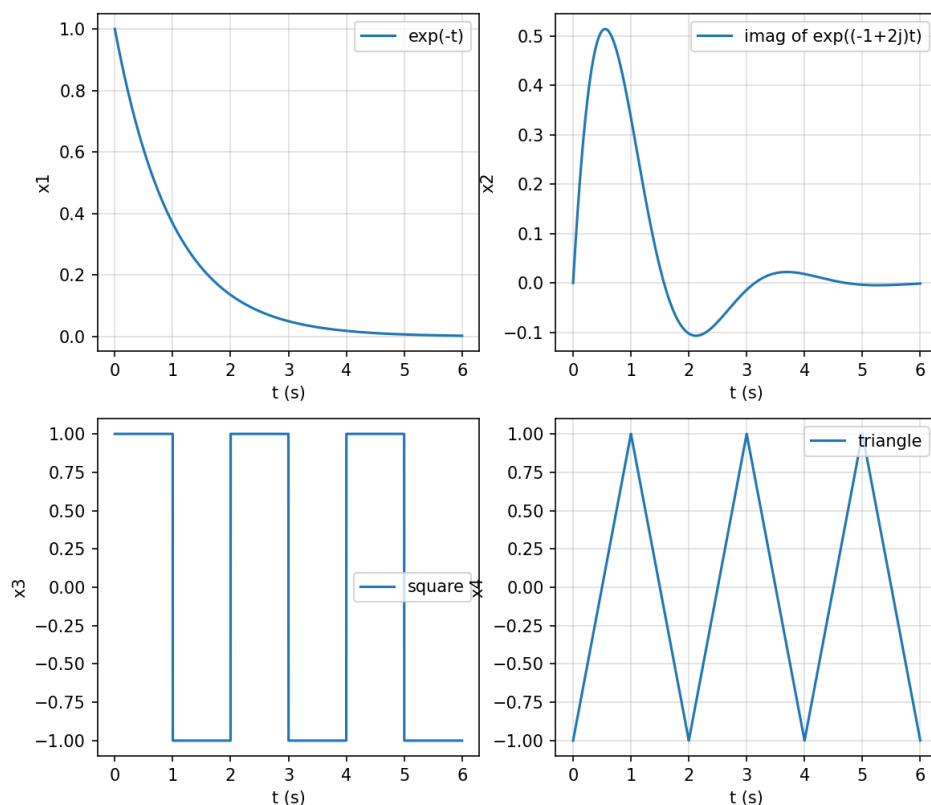


Figure 2: Continuous-time signals: real exponential, complex exponential (imaginary part), square wave, and triangular wave.

The code used to generate x_4 (triangular wave with $T = 2$):

```

1 T = 2
2 for i in range(len(t)):
3     rem = t[i] % T # tracks position of t within period
4     if rem < T/2:
5         x4[i] = -1 + (4/T) * rem # increasing portion of triangular
                                   wave
6     else:
7         x4[i] = 3 - (4/T) * rem # decreasing portion of triangular
                                   wave

```

Listing 1: Triangular wave generation

Question 1: Effect of a on the exponential

Changing a can affect the shape of the exponential in several different ways, depending on the sign of a . If $a > 0$, the function will exponentially grow to infinity, with larger magnitudes of a leading to faster exponential growth. If $a < 0$, the function will exponentially decay to 0, with larger magnitudes of a leading to faster decay. If $a = 0$, the exponential will equal 1 for all time.

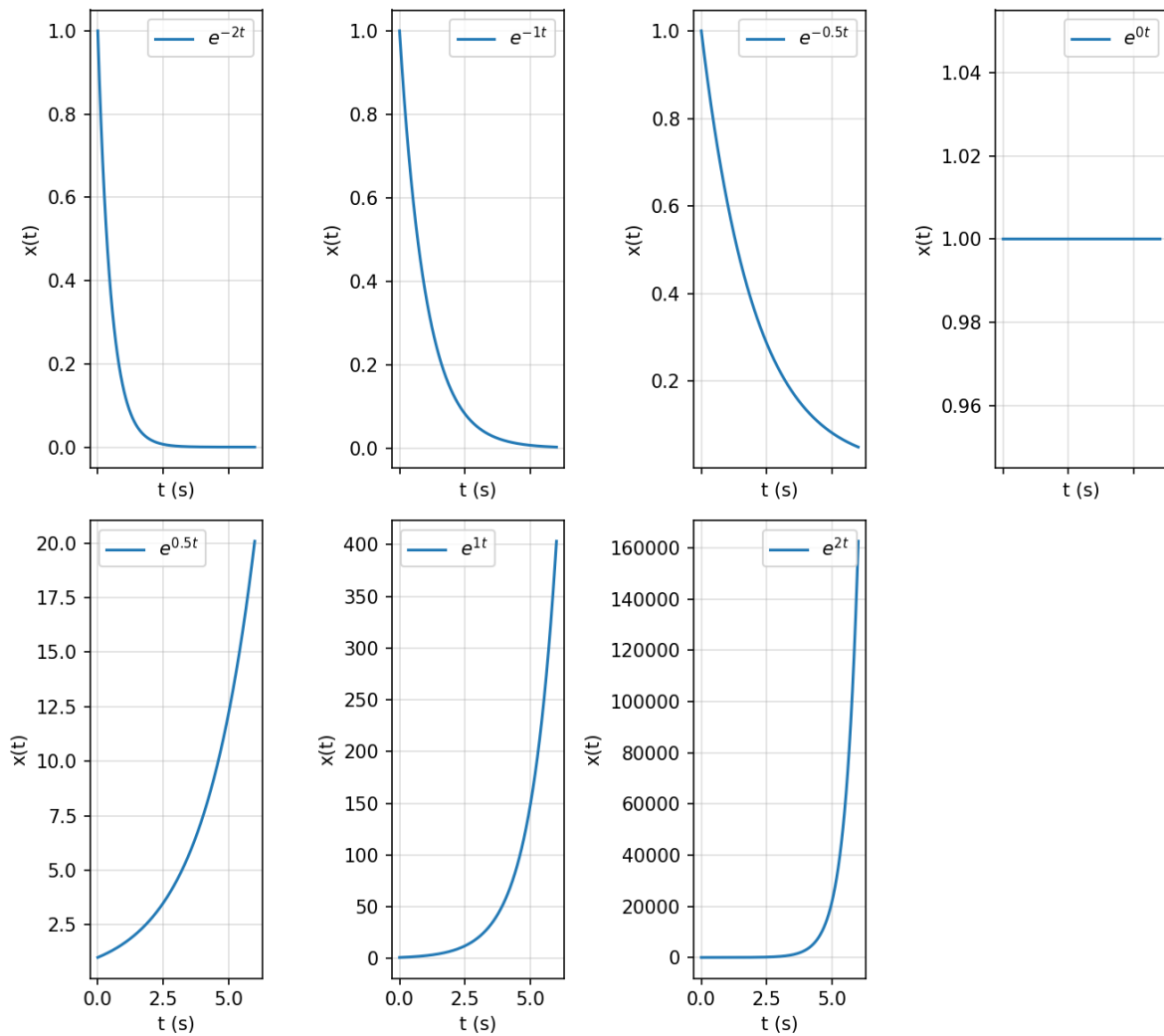


Figure 3: Real exponential e^{at} for several values of a .

Question 2: Envelope vs. oscillation rate in $e^{(\sigma+j\omega)t}$

In a complex exponential function $e^{(\sigma+j\omega)t}$, σ controls the envelope/amplitude, and ω sets the oscillation rate. This can be shown by splitting the function into two parts, the first equal to $e^{\sigma t}$, a real exponential function, and the second part equal to $e^{j\omega t}$, a complex exponential. The real exponential part of the function creates an envelope for the function, causing it to exponentially increase ($\sigma > 0$), or exponentially decay ($\sigma < 0$), as shown in Figure 3. However, if $\sigma = 0$, the real exponential function will equal 1 for all time and the behavior of the function will be entirely determined by the complex exponential portion of the function. By examining Figure 4, especially the 5th and 6th functions, where $\sigma = 0$, it can be determined that ω sets the oscillation rate, with higher values of ω leading to higher oscillation rates. This makes sense, because the complex exponential can be rewritten as $\cos(\omega t) + j\sin(\omega t)$, a periodic function with period $2\pi/\omega$ and frequency $\omega/2\pi$. As a result, higher values of ω will lead to shorter periods and faster frequencies/oscillation rates.

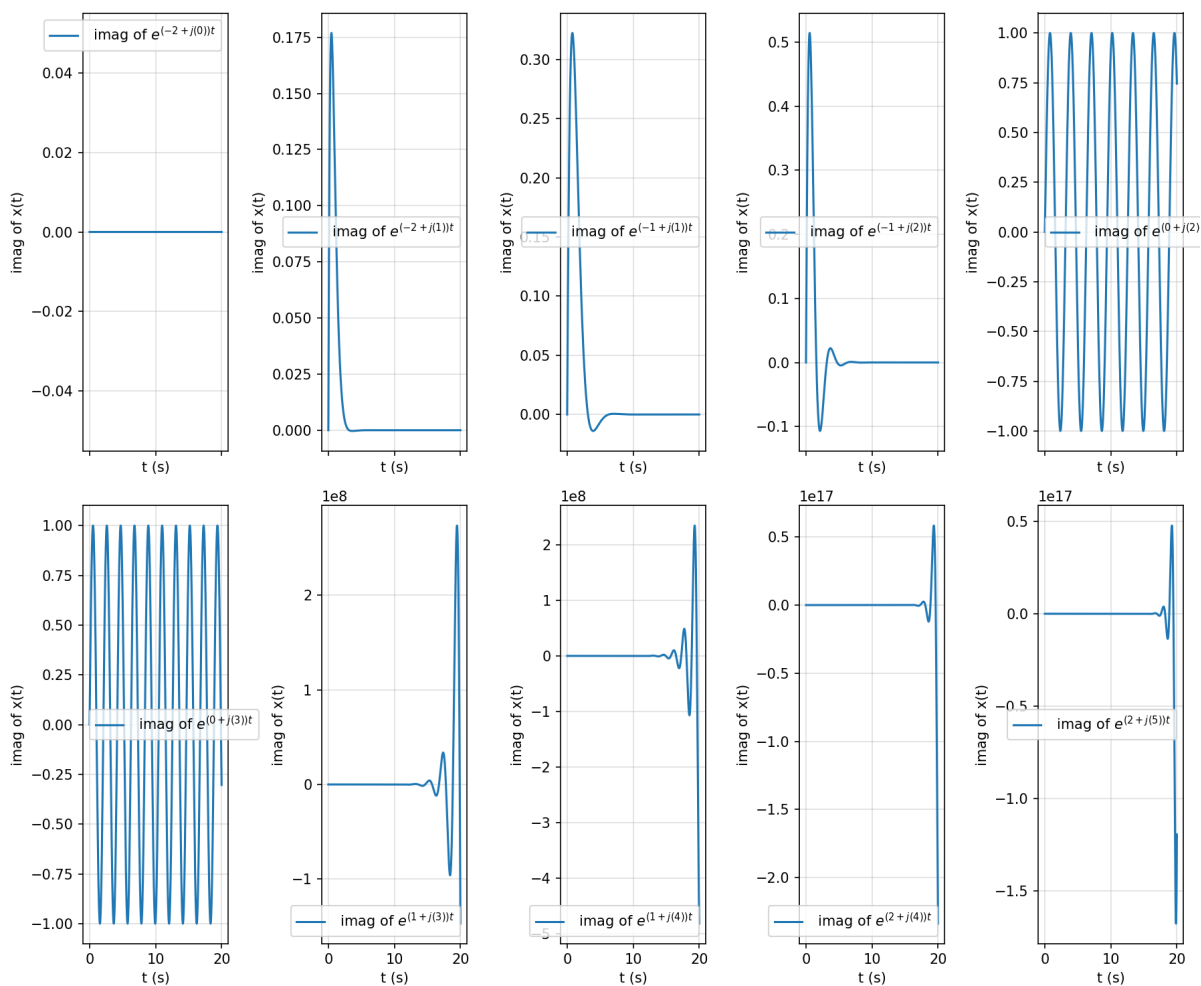


Figure 4: Complex exponential $e^{(\sigma+j\omega)t}$ with varied σ and ω .

Task 2.1

Hand-Drawn Solutions

Figure 5 shows the hand-drawn plots of $y_1[n] = x[n+2] + 1$, $y_2[n] = x[-n]$, $y_3[n] = x[2n]$ and $y_4[n] = 2x[-2n+1] - 1$, for $x[n] = n+1$, $0 \leq n \leq 5$.

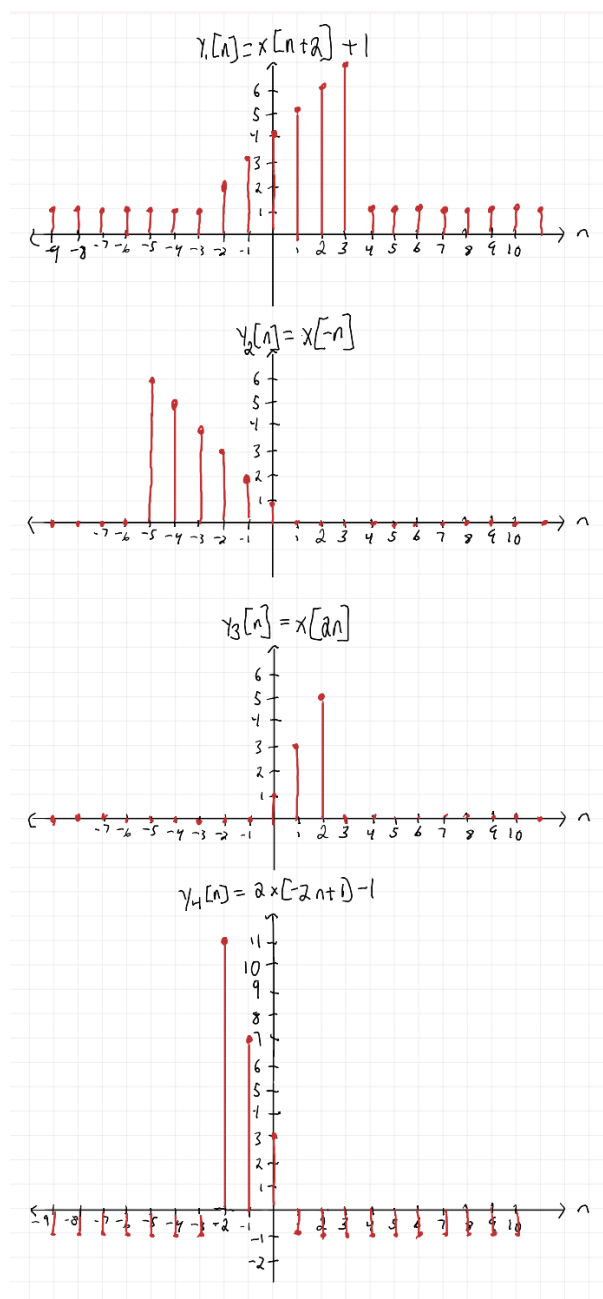


Figure 5: Hand-drawn plots of $y_1[n]$, $y_2[n]$, $y_3[n]$, $y_4[n]$

Stem Plots

Figure 6 shows the computer-plotted stem plots of $x[n+2]+1$, $x[-n]$, $x[2n]$ and $2x[-2n+1]-1$, for $x[n] = n+1, 0 \leq n \leq 5$.

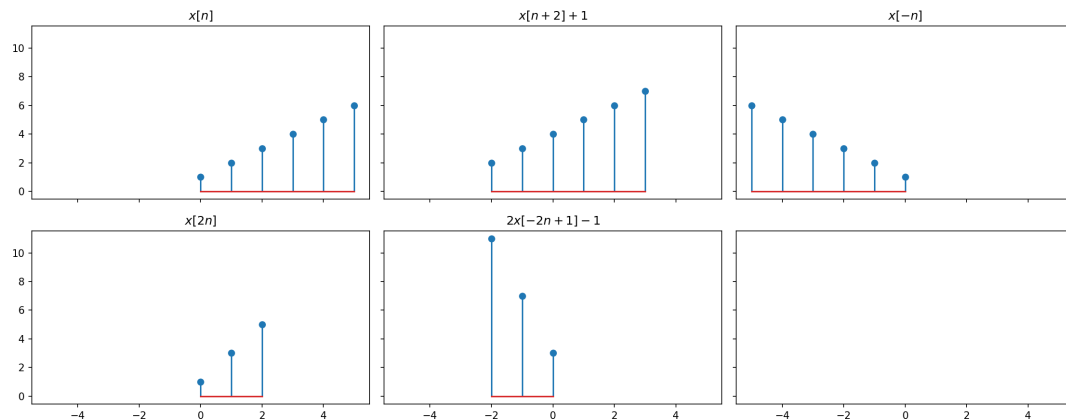


Figure 6: Discrete-time signal transformations: $x[n]$, $x[n+2]+1$, $x[-n]$, $x[2n]$, and $2x[-2n+1]-1$.

Question 1: Comparison of hand tables to stem plots

The hand-drawn plots match the computer plots for values of n that correspond to the original support of the function $x[n]$, $[0, 5]$ (though for the transformed function, this support has been moved around, such as in $y_1[n]$, where the support has been shifted to $[-2, 3]$). Additionally, the hand-drawn and computer plots for $y_2[n]$ and $y_3[n]$ match for all n . However, the hand-drawn plots and computer plots do not match outside of this transformed support for $y_1[n] = x[n+2]+1$ and $y_4[n] = 2x[-2n+1]-1$ (support = $[-2, 0]$), as the computer does not store any value for the signal outside of those n values, whereas in the hand-drawn plots, all other points are shown to have a vertical offset of +1 and -1 respectively.

Question 2: Samples retained under compression

$x[2n]$ retains 3 of the 6 samples when compared to $x[n]$. Compression cannot be undone by later expansion because the expansion will simply stretch the remaining samples further out, and there will be gaps in the signal anywhere that samples were lost from the original compression since the compressed signal no longer contains those values. For example, if the $x[2n]$ signal were to be expanded, the result would be $[1, 0, 3, 0, 5]$, instead of the original $[1, 2, 3, 4, 5, 6]$ as no information was retained of the discarded 2, 4, and 6 samples.

Question 3: $y_4[n] = 2x[-2n+1]-1$

The hand plot for $y_4[n] = 2x[-2n+1]-1$ can be seen in Figure 5 and the computer plot can be seen in Figure 6. They do not match, since the hand plot has a -1 vertical offset outside of $[-2, 0]$, whereas the computer has no signal stored for all values of n outside this range (similar to in $y_1[n]$). The code can be seen below, where the function is graphed without manually specifying the elements.

```
1 #  $y_4[n] = 2x[-2n+1] - 1$ 
2 x_sub = x[1::2]           #  $x[1], x[3], x[5] = [2, 4, 6]$ 
3 y4 = 2 * x_sub[::-1] - 1   # reverse so  $n$  runs  $-2 \rightarrow 0$ :  $[11, 7, 3]$ 
4 ny4 = np.arange(-(len(y4) - 1), 1) #  $[-2, -1, 0]$ 
```

Listing 2: Defining y_4

Task 3.1

The birthday tone was generated using $f = 1406$ Hz and sampled at $f_s = 8192$ Hz for $T = 4$ seconds.

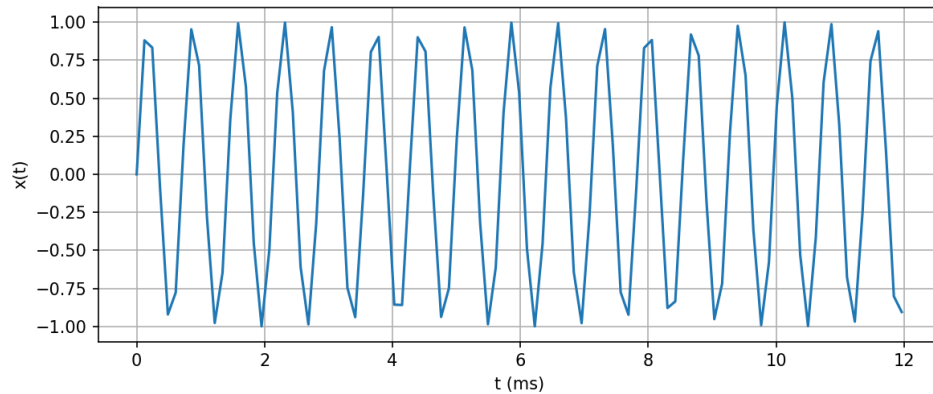


Figure 7: First 12 ms of the sampled sinusoid with $f = 1406$ Hz.

Question 1: Aliasing and the Nyquist rate

f_s was set to equal $c \cdot f$ and the sampling rate was varied by altering the value of c while keeping the original frequency f fixed. As c changed, the pitch of the played tone varied as follows:

- $c = 3.0$: matched the original
- $c = 2.1$: matched the original
- $c = 1.9$: lower
- $c = 1.5$: lower
- $c = 1.1$: lower

The pitch matches when $c > 2$ and drops when $c < 2$, with a threshold at $c = 2$, i.e. $f_s = 2f$. This is the Nyquist rate, and sampling at it only returns very low amplitude noise, as shown in Figure 10. Sampling above it ($f_s > 2f$) reproduces the tone correctly, whereas sampling below it ($f_s < 2f$) causes aliasing, where the tone folds down to a false lower frequency, causing the pitch to sound lower.

```

1 f = 1406 # birthday frequency tone
2
3 c = 2.00 # sampling rate multiplier, currently set to factor that
   produces the nyquist rate
4
5 fs = round(c*f) # sampling rate (Hz), as a multiple of the sampled
   frequency
6 T = 4.0 # duration (s)
7 N = int(fs * T) # total number of samples
8 n = np.arange(N) # sample index vector
9 t = n / fs # converts sample index to actual time
10 x = np.sin(2 * np.pi * f * t) # the sampled sinusoid

```

Listing 3: Determining Nyquist Rate

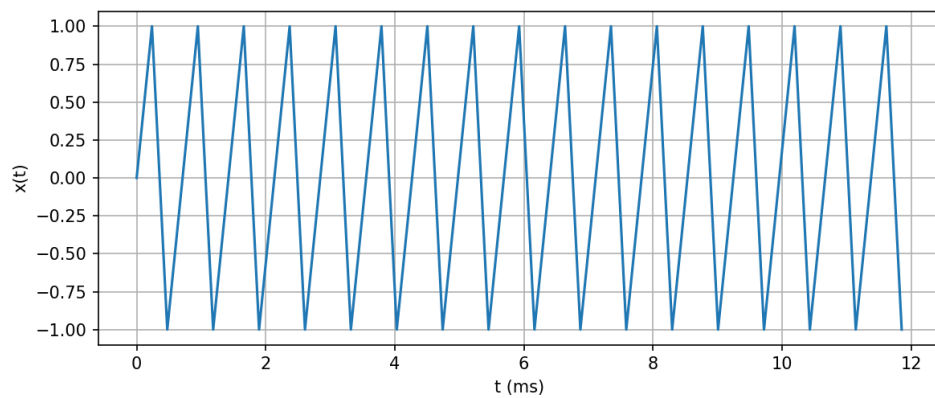


Figure 8: First 12 ms of the sampled sinusoid with $c = 3$ ($f_s = 3f$ Hz).

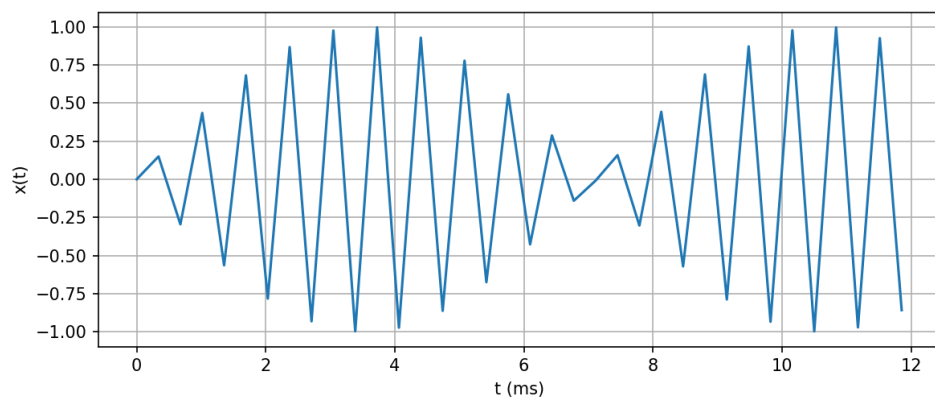


Figure 9: First 12 ms of the sampled sinusoid with $c = 2.1$ ($f_s = 2.1f$ Hz).

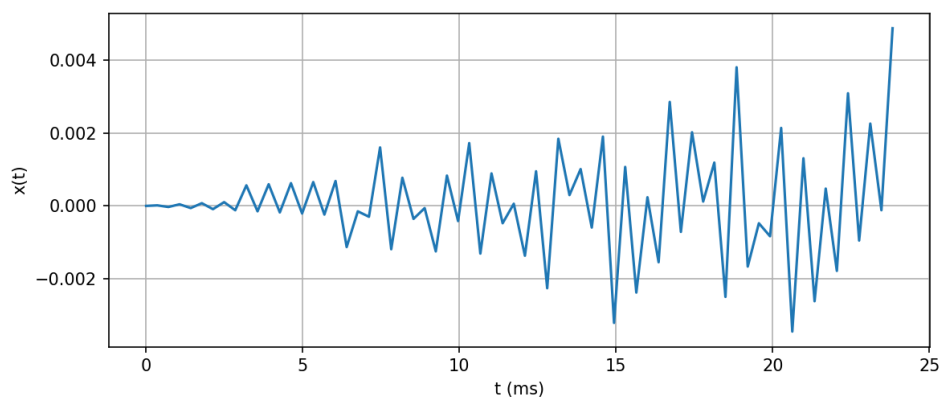


Figure 10: First 24 ms of the sampled sinusoid at Nyquist rate ($f_s = 2f$ Hz).

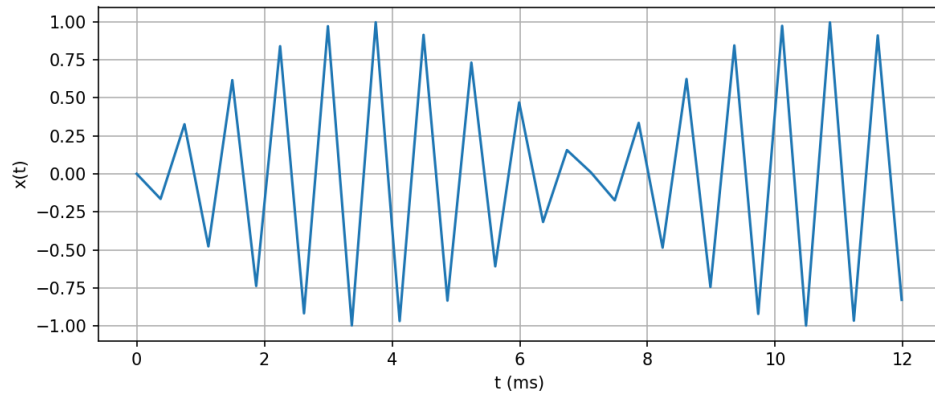


Figure 11: First 12 ms of the sampled sinusoid with $c = 1.9$ ($f_s = 1.9f$ Hz).

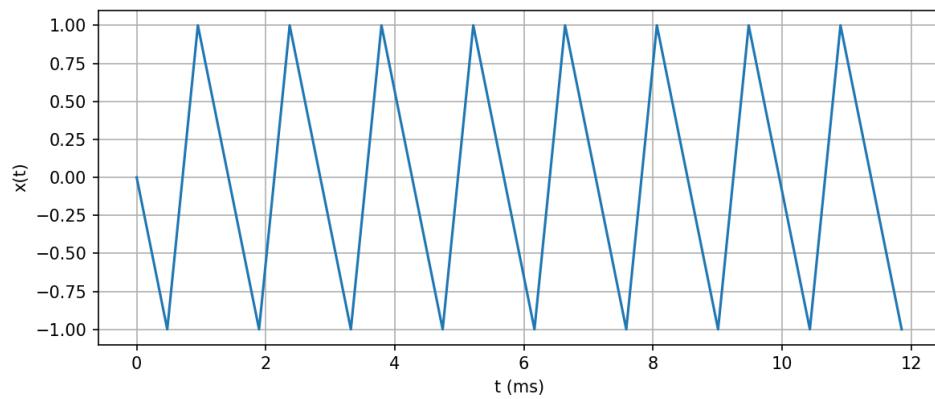


Figure 12: First 12 ms of the sampled sinusoid with $c = 1.5$ ($f_s = 1.5f$ Hz).

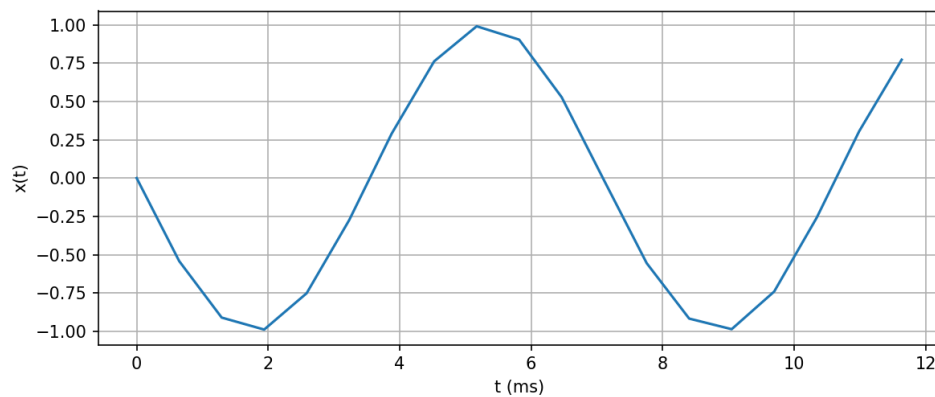


Figure 13: First 12 ms of the sampled sinusoid with $c = 1.1$ ($f_s = 1.1f$ Hz).

Question 2: Beat frequency

The sampling rate was reset to 8192 Hz, and an additional sinusoidal signal was added to the original tone signal with a slightly offset frequency of $\sin(2\pi(f + \Delta)t)$, making the total signal now equal to $\sin(2\pi ft) + \sin(2\pi(f + \Delta)t)$. This added an audible beating

pattern to the audio signal, which varied in period and rate for different values of Δ , as described below:

- $\Delta = 10$ Hz: beat period = 100 ms, beat rate = $1/0.1$ s = 10 Hz
- $\Delta = 5$ Hz: beat period = 200 ms, beat rate = $1/0.2$ s = 5 Hz
- $\Delta = 2$ Hz: beat period = 500 ms, beat rate = $1/0.5$ s = 2 Hz

In each case the beat rate is equal to Δ , in Hz, which makes sense mathematically via the sum-to-product identity for sin (Eq. 1):

$$\sin A + \sin B = 2 \sin\left(\frac{A+B}{2}\right) \cos\left(\frac{A-B}{2}\right). \quad (1)$$

Letting $A = 2\pi(f + \Delta)t$ and $B = 2\pi ft$:

$$\begin{aligned} & \sin(2\pi ft) + \sin(2\pi(f + \Delta)t) \\ &= 2 \sin\left(\frac{2\pi(f + \Delta)t + 2\pi ft}{2}\right) \cos\left(\frac{2\pi(f + \Delta)t - 2\pi ft}{2}\right) \\ &= 2 \sin(\pi(2f + \Delta)t) \cos(\pi\Delta t). \end{aligned}$$

This function is the product of two periodic signals with very different frequencies, the first equal to $\frac{2f+\Delta}{2}$ and the second equal to a much slower $\frac{\Delta}{2}$. An outer envelope is created by the slower signal, which has period $\frac{2}{\Delta}$ and completes a full cycle every $\frac{2}{\Delta}$ seconds. However, the magnitude of its amplitude is at its maximum twice as often, every $\frac{1}{2} \cdot \frac{2}{\Delta} = \frac{1}{\Delta}$ seconds, corresponding to audible peaks in loudness. Therefore, the signal beats every $\frac{1}{\Delta}$ seconds, with frequency or rate equal to $(\frac{1}{\Delta})^{-1} = \Delta$ Hz.

```

1 fs = 8192 # sampling rate (Hz)
2 T = 4.0 # duration (s)
3 N = int(fs * T) # total number of samples
4 n = np.arange(N) # sample index vector
5 t = n / fs # converts sample index to actual time
6 delta = 2 # offset of second sinusoid
7 x = np.sin(2 * np.pi * f * t) + np.sin(2 * np.pi * (f + delta) * t)
   # the sampled sinusoid, with additional added sinusoid offset by
   # delta
8
9 # plot x
10 x = x / np.max(np.abs(x)) # normalize to [-1, 1]
11 mask = t <= 1 # plot window widened to 1s
12 plt.figure(figsize=(8, 3.5))
13 plt.plot(t[mask] * 1e3, x[mask])
14 plt.xlabel("t (ms)"); plt.ylabel("x(t)")
15 plt.grid(True); plt.tight_layout()

```

Listing 4: Signal with Beat

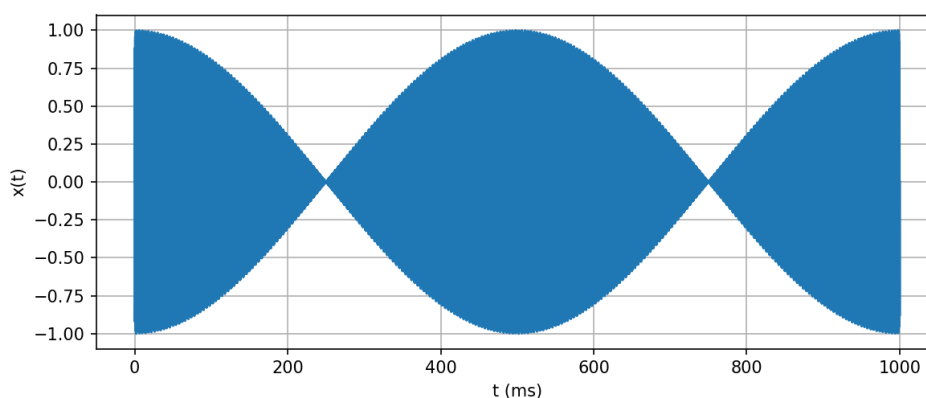


Figure 14: First 1 s of the combined sinusoid with beat rate of 2 Hz.

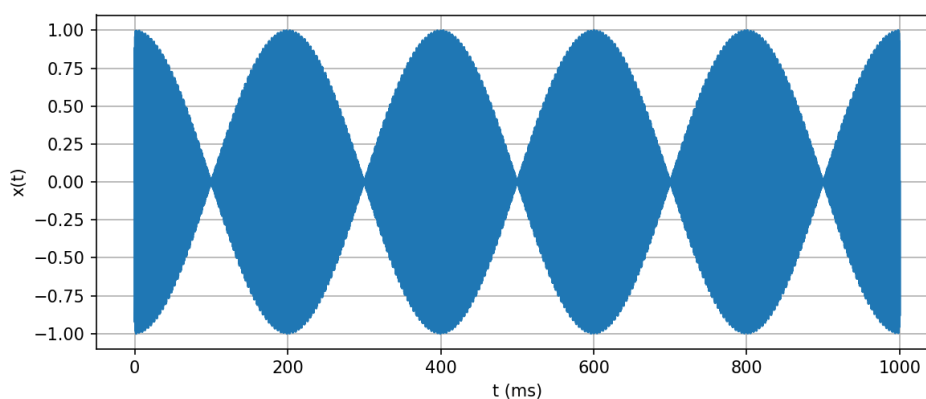


Figure 15: First 1 s of the combined sinusoid with beat rate of 5 Hz.

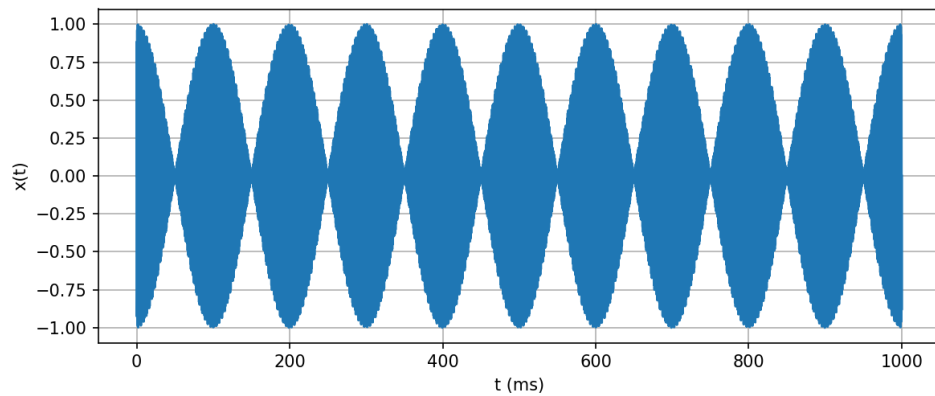


Figure 16: First 1 s of the combined sinusoid with beat rate of 10 Hz.

Question 3: Decaying envelope (plucked string)

When the original tone signal x was multiplied by the decaying envelope $e^{-t/\tau}$, with $\tau = 0.5$, the amplitude of the plotted signal decreased over time, as shown in Figure 17. This matched the audio that was heard by playing this signal, which became quieter over time until it was practically inaudible.

```

1 x = np.sin(2 * np.pi * f * t) # the sampled sinusoid
2 tau = 0.5 # decay factor
3 y = np.exp(-t/tau)*x # sampled sinusoid multiplied by decaying
  envelope
4
5 # plot y
6 y = y / np.max(np.abs(y)) # normalize to [-1, 1]
7 mask = t <= 1 # plot first 1 s of signal
8 plt.figure(figsize=(8, 3.5))
9 plt.plot(t[mask] * 1e3, y[mask])
10 plt.xlabel("t (ms)"); plt.ylabel("y(t)")
11 plt.grid(True); plt.tight_layout()

```

Listing 5: Signal with Decaying Envelope

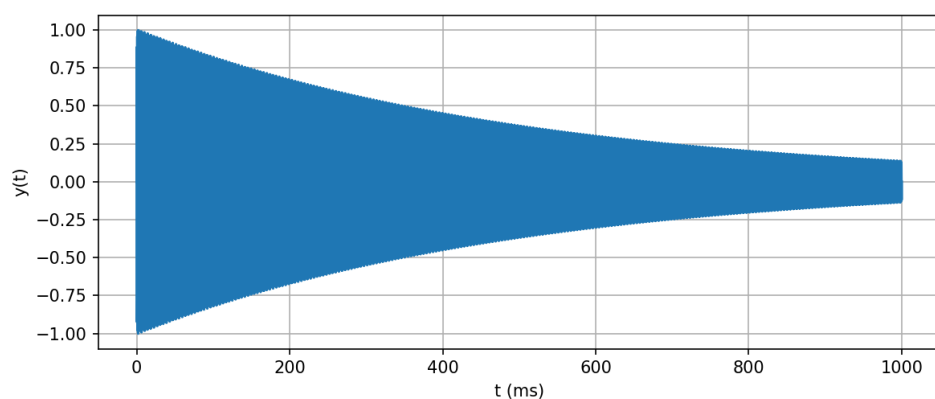


Figure 17: First 1 s of signal with decaying envelope.

Conclusion

This lab was all about getting comfortable with using and plotting signals in python and observing how the math behind the signals actually shows up in plots and sound. In the first part we found that in the real exponential function e^{at} , a sets whether the function exponentially grows or decays, and that in the complex exponential $e^{(\sigma+j\omega)t}$, σ controls the envelope of the function while ω controls how fast it oscillates. In the second part we saw that hand plots and computer plots can differ, since the computer only stores signal values for n within a small finite range, and that compressing a signal throws away samples that cannot be regained. In the third part, we found that sampling a signal below the Nyquist rate $f_s = 2f$ causes aliasing, making the original frequency sound at a lower pitch than it actually was. We also found that the combination of sinusoids with slightly offset frequencies produces a beat at a rate of Δ , which was mathematically explained by the sum-to-product identity. Lastly, we found that multiplying a signal by a decaying envelope made the tone imitate that of a plucked string by decreasing its amplitude over time until the sound became inaudible.

The biggest thing we learned in this lab is how what we hear and see can be explained straight from the math, such as how beats are the product of sinusoids, as well as the importance of sampling higher than the Nyquist rate to avoid aliasing the signal. Additionally, the exercise involving beats showed that adding two very similar sinusoids can make something that looks and sounds completely different than expected.